Summary report

Muslim,Dinmukhammed

1. Introduction

This document presents the joint comparison report for Assignment 2 (Pair 2).
The purpose of this summary is to analyze and compare two sorting algorithms implemented by the pair: **Heap Sort** and **Shell Sort**.
Both algorithms were implemented in Java, tested on identical input data, and evaluated using the same performance metrics such as time, comparisons, array accesses, and moves.

2. Algorithm Overview

Heap Sort
Heap Sort is a comparison-based sorting algorithm that organizes data into a binary heap structure.
It repeatedly extracts the largest element (for max-heap) and rebuilds the heap until all elements are sorted.
Heap Sort has a guaranteed time complexity of **O(n log n)** in all cases, uses **O(1)** space, and is not stable.

Shell Sort
Shell Sort is an optimization of Insertion Sort that allows elements far apart to be compared and moved earlier in the process.
It works by sorting elements separated by a gap sequence that gradually decreases to 1.
Its performance depends heavily on the chosen gap sequence, with average complexity between **O(n1.25)** and **O(n1.5)**.

3. Theoretical Comparison

Both algorithms are in-place, comparison-based, and not stable.
Heap Sort provides predictable O(n log n) performance across all cases,
while Shell Sort's performance can vary depending on the input and the gap sequence used.

Theoretical Summary:

- Heap Sort: consistent O(n log n) performance, guaranteed for all inputs.
- Shell Sort: can be faster for small or nearly sorted arrays but may degrade to $O(n^2)$.

4. Empirical Comparison

Both algorithms were tested on the same dataset sizes: **100**, **1000**, and **10,000** elements.
Input distributions included **random**, **sorted**, **reversed**, and **nearly sorted** arrays.

Heap Sort Results (by Muslim):
Heap Sort showed consistent logarithmic growth in execution time as input size increased.
The algorithm performed fastest on nearly sorted data and slightly slower on reversed arrays.
Performance remained predictable and stable for all distributions.

Shell Sort Results (by Dinmukhamed):
Shell Sort demonstrated faster execution on small datasets due to fewer operations per element.
However, its runtime increased more rapidly as n grew larger, especially on random or reversed data.
The choice of gap sequence had a noticeable impact on performance stability.

General Observation:
For small arrays, Shell Sort can outperform Heap Sort.
For large datasets, Heap Sort becomes clearly more efficient due to its O(n log n) scaling.

5. Discussion

Heap Sort Advantages:

- Predictable and consistent O(n log n) runtime.
- Works efficiently for large datasets.
- Low memory usage (O(1)).

Heap Sort Disadvantages:

- Slightly slower for small arrays.
- Implementation is more complex.

Shell Sort Advantages:

- Very simple implementation.
- Performs well on small or partially sorted arrays.

Shell Sort Disadvantages:

- Performance highly dependent on gap sequence.
- Scales poorly for large datasets.
- May degrade to $O(n^2)$ in the worst case.

Overall, Heap Sort is more suitable for **large datasets** and tasks requiring predictable performance,
while Shell Sort is better for **small or nearly sorted data** due to lower constant factors.

6. Conclusion

The comparison between Heap Sort and Shell Sort shows that both algorithms are efficient,
but they are optimized for different use cases.
Heap Sort guarantees stable O(n log n) performance regardless of input distribution,
making it ideal for large-scale sorting.
Shell Sort, while less predictable, is simpler and can outperform Heap Sort for smaller datasets.

In conclusion, **Heap Sort** is recommended for large datasets requiring consistent performance,
and **Shell Sort** is more practical for small or nearly sorted arrays where quick passes are sufficient.